# **Música** Solución

*Academia de Música SonoRitmo — Solución propuesta en Java 21 + JavaFX 21 + Gson*

## Diseño de la solución

El modelo entregado (`ClaseMusical`, `ClaseIndividual`, `ClaseGrupal`, `ConSalaPrivada`) **no se modifica**. Sobre él se agregan tres capas, cada una con una sola responsabilidad:

| Capa | Clases | Responsabilidad |
|---|---|---|
| Vista | `view/main-view.fxml` | Declara el formulario, la tabla, los botones y el área de mensajes. No contiene lógica. |
| Controlador | `controller.MainController` | Valida el formulario, arma el objeto del modelo, pide al repositorio guardar/eliminar y actualiza la interfaz. |
| Repository | `repository.ClaseMusicalRepository` | Mantiene la colección en memoria, guarda tras cada cambio y deshace si falla. |
| DAO | `dao.Dao`, `dao.JsonDao`, `dao.PersistenciaException` | Lee y escribe JSON con Gson. No conoce JavaFX. |

**Cálculo de costo**

Clase Individual: costo base de $60.000. Si el estudiante no rindió una evaluación de nivel, el costo se incrementa en un 15%.
Clase Grupal: costo base de $25.000. Si la cantidad de integrantes supera 6, el costo se incrementa en un 10%.
Ya lo resuelve el modelo: la tabla solo invoca `calcularCosto()`.

**Cómo ejecutarla**

Desde la carpeta del proyecto: `mvn javafx:run` (o ejecutar `Launcher` desde IntelliJ). Los archivos JSON se crean en la carpeta desde la que se ejecuta.

## Cambios y archivos nuevos

`pom.xml`

Único cambio: la dependencia de Gson, en el lugar del comentario.

```
        <!-- Gson: lectura y escritura de archivos JSON -->
        <dependency>
            <groupId>com.google.code.gson</groupId>
            <artifactId>gson</artifactId>
            <version>2.11.0</version>
        </dependency>
```

`dao/Dao.java`

Contrato de lectura y escritura. Es genérico para reutilizarlo con ambos subtipos.

```java
package cl.duoc.sonoritmo.dao;

import java.util.List;

/**
 * Contrato de acceso a datos: lectura y escritura de una colección de objetos de dominio.
 *
 * @param <T> tipo de objeto que se almacena
 */
public interface Dao<T> {

    /**
     * Lee todos los objetos almacenados.
     *
     * @return lista de objetos; vacía si aún no hay datos
     * @throws PersistenciaException si el medio de almacenamiento no se puede leer
     */
    List<T> cargar();

    /**
     * Reemplaza el contenido almacenado por la lista recibida.
     *
     * @param datos objetos que deben quedar almacenados
     * @throws PersistenciaException si el medio de almacenamiento no se puede escribir
     */
    void guardar(List<T> datos);
}
```

`dao/PersistenciaException.java`

Error propio, no comprobado, para fallas de lectura o escritura.

```java
package cl.duoc.sonoritmo.dao;

/**
 * Error al leer o escribir el medio de almacenamiento.
 */
public class PersistenciaException extends RuntimeException {

    public PersistenciaException(String mensaje, Throwable causa) {
        super(mensaje, causa);
    }
}
```

`dao/JsonDao.java`

Implementación con Gson. Si el archivo no existe devuelve una lista vacía (primera ejecución sin errores).

```java
package cl.duoc.sonoritmo.dao;

import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import com.google.gson.JsonParseException;
import com.google.gson.reflect.TypeToken;

import java.io.IOException;
import java.io.Reader;
import java.io.Writer;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

/**
 * Implementación de {@link Dao} que guarda los objetos en un archivo JSON usando Gson.
 *
```

```
 * @param <T> tipo de objeto que se almacena
 */
public class JsonDao<T> implements Dao<T> {

    private final Path archivo;
    private final Class<T> tipo;
    private final Gson gson = new GsonBuilder().setPrettyPrinting().create();

    public JsonDao(String nombreArchivo, Class<T> tipo) {
        this.archivo = Path.of(nombreArchivo);
        this.tipo = tipo;
    }

    @Override
    public List<T> cargar() {
        if (!Files.exists(archivo)) {
            return new ArrayList<>();
        }
        try (Reader lector = Files.newBufferedReader(archivo, StandardCharsets.UTF_8)) {
            List<T> datos = gson.fromJson(lector, TypeToken.getParameterized(List.class, tipo).getType());
            return datos == null ? new ArrayList<>() : datos;
        } catch (IOException | JsonParseException e) {
            throw new PersistenciaException("No se pudo leer el archivo " + archivo.getFileName() + ".",
e);
        }
    }

    @Override
    public void guardar(List<T> datos) {
        try (Writer escritor = Files.newBufferedWriter(archivo, StandardCharsets.UTF_8)) {
            gson.toJson(datos, TypeToken.getParameterized(List.class, tipo).getType(), escritor);
        } catch (IOException | JsonParseException e) {
            throw new PersistenciaException("No se pudo guardar el archivo " + archivo.getFileName() + ".",
e);
        }
    }
}
```

`repository/ClaseMusicalRepository.java`

Reúne los dos DAO en una sola colección de `ClaseMusical`. `agregar` y `eliminar` guardan al instante, de modo que memoria y archivos coinciden.

```
package cl.duoc.sonoritmo.repository;

import cl.duoc.sonoritmo.dao.Dao;
import cl.duoc.sonoritmo.model.ClaseMusical;
import cl.duoc.sonoritmo.model.ClaseIndividual;
import cl.duoc.sonoritmo.model.ClaseGrupal;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Centraliza las operaciones sobre la colección de clases y delega en los DAO
 * la lectura y escritura. Cada cambio se guarda de inmediato, de modo que la memoria
 * y los archivos siempre coinciden.
 */
public class ClaseMusicalRepository {

    private final Dao<ClaseIndividual> daoClaseIndividual;
    private final Dao<ClaseGrupal> daoClaseGrupal;
    private final List<ClaseMusical> clases = new ArrayList<>();

    public ClaseMusicalRepository(Dao<ClaseIndividual> daoClaseIndividual, Dao<ClaseGrupal> daoClaseGrupal)
{
        this.daoClaseIndividual = daoClaseIndividual;
        this.daoClaseGrupal = daoClaseGrupal;
    }
```

```
    /**
     * Carga en memoria los datos guardados en los archivos.
     */
    public void cargar() {
        clases.clear();
        clases.addAll(daoClaseIndividual.cargar());
        clases.addAll(daoClaseGrupal.cargar());
    }

    /**
     * @return vista de solo lectura de la colección
     */
    public List<ClaseMusical> listar() {
        return Collections.unmodifiableList(clases);
    }

    /**
     * Agrega un elemento y guarda los archivos. Si no se puede guardar, deja la colección como estaba.
     */
    public void agregar(ClaseMusical nuevo) {
        clases.add(nuevo);
        try {
            guardar();
        } catch (RuntimeException e) {
            clases.remove(nuevo);
            throw e;
        }
    }

    /**
     * Elimina un elemento y guarda los archivos. Si no se puede guardar, deja la colección como estaba.
     */
    public void eliminar(ClaseMusical existente) {
        int posicion = clases.indexOf(existente);
        if (posicion < 0) {
            return;
        }
        clases.remove(posicion);
        try {
            guardar();
        } catch (RuntimeException e) {
            clases.add(posicion, existente);
            throw e;
        }
    }

    private void guardar() {
        List<ClaseIndividual> lista1 = new ArrayList<>();
        List<ClaseGrupal> lista2 = new ArrayList<>();
        for (ClaseMusical elemento : clases) {
            if (elemento instanceof ClaseIndividual a) {
                lista1.add(a);
            } else if (elemento instanceof ClaseGrupal b) {
                lista2.add(b);
            }
        }
        daoClaseIndividual.guardar(lista1);
        daoClaseGrupal.guardar(lista2);
    }
}
```

`view/main-view.fxml`

Vista declarada en FXML, vinculada al controlador con `fx:controller`. Cada control tiene su `fx:id`.

```
<?xml version="1.0" encoding="UTF-8"?>

<?import java.lang.String?>
<?import javafx.collections.FXCollections?>
<?import javafx.geometry.Insets?>
```

```
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.CheckBox?>
<?import javafx.scene.control.ComboBox?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.TableColumn?>
<?import javafx.scene.control.TableView?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.ColumnConstraints?>
<?import javafx.scene.layout.GridPane?>
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.layout.VBox?>

<VBox spacing="10.0" prefWidth="760.0" xmlns="http://javafx.com/javafx/21"
xmlns:fx="http://javafx.com/fxml/1" fx:controller="cl.duoc.sonoritmo.controller.MainController">
   <padding>
      <Insets bottom="15.0" left="15.0" right="15.0" top="15.0" />
   </padding>
   <Label text="SonoRitmo: registro de clases" style="-fx-font-size: 16px; -fx-font-weight: bold;" />
   <GridPane hgap="10.0" vgap="8.0">
      <columnConstraints>
         <ColumnConstraints minWidth="150.0" />
         <ColumnConstraints hgrow="ALWAYS" minWidth="150.0" />
         <ColumnConstraints minWidth="150.0" />
         <ColumnConstraints hgrow="ALWAYS" minWidth="150.0" />
      </columnConstraints>
      <Label text="Tipo" GridPane.columnIndex="0" GridPane.rowIndex="0" />
      <ComboBox fx:id="cbTipo" prefWidth="260.0" promptText="Seleccione un tipo" GridPane.columnIndex="1"
GridPane.columnSpan="2" GridPane.rowIndex="0">
         <items>
            <FXCollections fx:factory="observableArrayList">
               <String fx:value="Clase Individual" />
               <String fx:value="Clase Grupal" />
            </FXCollections>
         </items>
      </ComboBox>
      <Label text="Código de clase" GridPane.columnIndex="0" GridPane.rowIndex="1" />
      <TextField fx:id="txtCodigoClase" GridPane.columnIndex="1" GridPane.rowIndex="1" />
      <Label text="Duración (meses)" GridPane.columnIndex="2" GridPane.rowIndex="1" />
      <TextField fx:id="txtDuracionMeses" GridPane.columnIndex="3" GridPane.rowIndex="1" />
      <Label text="Cupo máximo" GridPane.columnIndex="0" GridPane.rowIndex="2" />
      <TextField fx:id="txtCupoMaximo" GridPane.columnIndex="1" GridPane.rowIndex="2" />
      <Label text="Datos de la clase individual" GridPane.columnIndex="0" GridPane.rowIndex="3"
GridPane.columnSpan="4" style="-fx-font-weight: bold;" />
      <Label text="Instrumento principal" GridPane.columnIndex="0" GridPane.rowIndex="4" />
      <TextField fx:id="txtInstrumento" GridPane.columnIndex="1" GridPane.rowIndex="4" />
      <CheckBox fx:id="chkEvaluacionNivel" mnemonicParsing="false" text="Rindió evaluación de nivel"
GridPane.columnIndex="2" GridPane.columnSpan="2" GridPane.rowIndex="4" />
      <CheckBox fx:id="chkSalaPrivada" mnemonicParsing="false" text="Sala de ensayo privada asignada"
GridPane.columnIndex="0" GridPane.columnSpan="2" GridPane.rowIndex="5" />
      <Label text="Datos de la clase grupal" GridPane.columnIndex="0" GridPane.rowIndex="6"
GridPane.columnSpan="4" style="-fx-font-weight: bold;" />
      <Label text="Integrantes" GridPane.columnIndex="0" GridPane.rowIndex="7" />
      <TextField fx:id="txtIntegrantes" GridPane.columnIndex="1" GridPane.rowIndex="7" />
   </GridPane>
   <HBox spacing="10.0">
      <Button fx:id="btnRegistrar" mnemonicParsing="false" onAction="#onRegistrar" text="Registrar" />
      <Button fx:id="btnEliminar" mnemonicParsing="false" onAction="#onEliminar" text="Eliminar" />
   </HBox>
   <TableView fx:id="tabla" prefHeight="230.0">
      <columns>
         <TableColumn fx:id="colTipo" text="Tipo" />
         <TableColumn fx:id="colCodigoClase" text="Código" />
         <TableColumn fx:id="colDuracionMeses" text="Duración (meses)" />
         <TableColumn fx:id="colCupoMaximo" text="Cupo" />
         <TableColumn fx:id="colCosto" text="Costo" />
      </columns>
      <columnResizePolicy>
         <TableView fx:constant="CONSTRAINED_RESIZE_POLICY" />
      </columnResizePolicy>
   </TableView>
   <Label fx:id="lblMensaje" wrapText="true" minHeight="40.0" maxWidth="Infinity" />
</VBox>
```

`controller/MainController.java`

Controlador: `initialize()` configura columnas, carga datos y escucha el selector; `onRegistrar` y `onEliminar` son los manejadores de eventos.

```
package cl.duoc.sonoritmo.controller;

import cl.duoc.sonoritmo.dao.JsonDao;
import cl.duoc.sonoritmo.dao.PersistenciaException;
import cl.duoc.sonoritmo.model.ClaseMusical;
import cl.duoc.sonoritmo.model.ClaseIndividual;
import cl.duoc.sonoritmo.model.ClaseGrupal;
import cl.duoc.sonoritmo.repository.ClaseMusicalRepository;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.CheckBox;
import javafx.scene.control.ComboBox;
import javafx.scene.control.Label;
import javafx.scene.control.ListCell;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.TextField;

import java.text.DecimalFormat;
import java.text.DecimalFormatSymbols;
import java.util.Locale;


/**
 * Controlador de la vista principal: coordina el formulario, la tabla y los mensajes.
 * La lógica de negocio vive en el modelo y el acceso a datos en el repositorio.
 */
public class MainController {

    private static final String TIPO_CLASE_INDIVIDUAL = "Clase Individual";
    private static final String TIPO_CLASE_GRUPAL = "Clase Grupal";
    private static final Locale CHILE = Locale.forLanguageTag("es-CL");
    private static final DecimalFormat FORMATO_MONEDA = new DecimalFormat("$#,##0",
DecimalFormatSymbols.getInstance(CHILE));
    private static final String COLOR_OK = "-fx-text-fill: #1b5e20;";
    private static final String COLOR_ERROR = "-fx-text-fill: #b71c1c;";

    @FXML private ComboBox<String> cbTipo;
    @FXML private TextField txtCodigoClase;
    @FXML private TextField txtDuracionMeses;
    @FXML private TextField txtCupoMaximo;
    @FXML private TextField txtInstrumento;
    @FXML private CheckBox chkEvaluacionNivel;
    @FXML private CheckBox chkSalaPrivada;
    @FXML private TextField txtIntegrantes;
    @FXML private TableView<ClaseMusical> tabla;
    @FXML private TableColumn<ClaseMusical, String> colTipo;
    @FXML private TableColumn<ClaseMusical, String> colCodigoClase;
    @FXML private TableColumn<ClaseMusical, String> colDuracionMeses;
    @FXML private TableColumn<ClaseMusical, String> colCupoMaximo;
    @FXML private TableColumn<ClaseMusical, String> colCosto;
    @FXML private Label lblMensaje;

    private final ClaseMusicalRepository repositorio = new ClaseMusicalRepository(
            new JsonDao<>("clases_individuales.json", ClaseIndividual.class),
            new JsonDao<>("clases_grupales.json", ClaseGrupal.class));
    private final ObservableList<ClaseMusical> datosTabla = FXCollections.observableArrayList();

    @FXML
    public void initialize() {
        // El selector muestra el texto de ayuda también cuando no hay selección.
        cbTipo.setButtonCell(new ListCell<>() {
            @Override
```

```
            protected void updateItem(String item, boolean vacio) {
                super.updateItem(item, vacio);
                setText(vacio || item == null ? cbTipo.getPromptText() : item);
            }
        });
        cbTipo.valueProperty().addListener((obs, anterior, nuevo) -> actualizarCamposPorTipo());

        colTipo.setCellValueFactory(celda -> new SimpleStringProperty(celda.getValue().getTipo()));
        colCodigoClase.setCellValueFactory(celda -> new
SimpleStringProperty(celda.getValue().getCodigoClase()));
        colDuracionMeses.setCellValueFactory(celda -> new
SimpleStringProperty(String.valueOf(celda.getValue().getDuracionMeses())));
        colCupoMaximo.setCellValueFactory(celda -> new
SimpleStringProperty(String.valueOf(celda.getValue().getCupoMaximo())));
        colCosto.setCellValueFactory(celda -> new SimpleStringProperty(
                FORMATO_MONEDA.format(celda.getValue().calcularCosto())));
        tabla.setItems(datosTabla);

        actualizarCamposPorTipo();
        try {
            repositorio.cargar();
        } catch (PersistenciaException e) {
            mostrarError(e.getMessage());
        }
        refrescarTabla();
    }

    @FXML
    private void onRegistrar() {
        try {
            ClaseMusical nuevo = construirDesdeFormulario();
            repositorio.agregar(nuevo);
            refrescarTabla();
            mostrarExito(nuevo.getCodigoClase() + " (" + nuevo.getTipo() + ") registrada correctamente.");
            limpiarFormulario();
        } catch (IllegalArgumentException | PersistenciaException e) {
            mostrarError(e.getMessage());
        }
    }

    @FXML
    private void onEliminar() {
        ClaseMusical seleccionado = tabla.getSelectionModel().getSelectedItem();
        if (seleccionado == null) {
            mostrarError("Debe seleccionar una clase de la tabla para eliminarla.");
            return;
        }
        try {
            repositorio.eliminar(seleccionado);
            refrescarTabla();
            mostrarExito(seleccionado.getCodigoClase() + " (" + seleccionado.getTipo() + ") eliminada
correctamente.");
        } catch (PersistenciaException e) {
            mostrarError(e.getMessage());
        }
    }

    /**
     * Valida el formulario y construye el objeto del tipo elegido. Lanza
     * IllegalArgumentException con el campo y el motivo si algún dato no es válido.
     */
    private ClaseMusical construirDesdeFormulario() {
        String tipo = cbTipo.getValue();
        if (tipo == null) {
            throw new IllegalArgumentException("Tipo: debe seleccionar un tipo de clase.");
        }
        String codigoClase = leerTexto(txtCodigoClase, "Código de clase");
        int duracionMeses = leerEntero(txtDuracionMeses, "Duración (meses)", 1, 12, "debe estar entre 1 y
12.");
        int cupoMaximo = leerEntero(txtCupoMaximo, "Cupo máximo", 1, Integer.MAX_VALUE, "debe ser mayor que
cero.");
```

```
        if (TIPO_CLASE_INDIVIDUAL.equals(tipo)) {
            String instrumento = leerTexto(txtInstrumento, "Instrumento principal");
            ClaseIndividual claseIndividual = new ClaseIndividual(codigoClase, duracionMeses, cupoMaximo,
instrumento, chkEvaluacionNivel.isSelected());
            if (chkSalaPrivada.isSelected()) {
                claseIndividual.asignarSalaPrivada();
            }
            return claseIndividual;
        }

        // Segundo tipo: Clase Grupal
        int integrantes = leerEntero(txtIntegrantes, "Integrantes", 1, Integer.MAX_VALUE, "debe ser mayor
que cero.");
        ClaseGrupal claseGrupal = new ClaseGrupal(codigoClase, duracionMeses, cupoMaximo, integrantes);
        return claseGrupal;
    }

    private String leerTexto(TextField campo, String nombre) {
        String valor = campo.getText() == null ? "" : campo.getText().trim();
        if (valor.isEmpty()) {
            throw new IllegalArgumentException(nombre + ": no puede estar vacío ni contener solo
espacios.");
        }
        return valor;
    }

    private int leerEntero(TextField campo, String nombre, int minimo, int maximo, String regla) {
        int valor;
        try {
            valor = Integer.parseInt(campo.getText().trim());
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException(nombre + ": debe ser un número entero.");
        }
        if (valor < minimo || valor > maximo) {
            throw new IllegalArgumentException(nombre + ": " + regla);
        }
        return valor;
    }

    private double leerDecimalPositivo(TextField campo, String nombre) {
        double valor;
        try {
            valor = Double.parseDouble(campo.getText().trim().replace(',', '.'));
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException(nombre + ": debe ser un número (se aceptan decimales).");
        }
        if (valor <= 0) {
            throw new IllegalArgumentException(nombre + ": debe ser mayor que cero.");
        }
        return valor;
    }

    /** Habilita solo los datos propios del tipo elegido; los demás se deshabilitan y se vacían. */
    private void actualizarCamposPorTipo() {
        String tipo = cbTipo.getValue();
        boolean esClaseIndividual = TIPO_CLASE_INDIVIDUAL.equals(tipo);
        boolean esClaseGrupal = TIPO_CLASE_GRUPAL.equals(tipo);
        txtInstrumento.setDisable(!esClaseIndividual);
        if (!esClaseIndividual) {
            txtInstrumento.clear();
        }
        chkEvaluacionNivel.setDisable(!esClaseIndividual);
        if (!esClaseIndividual) {
            chkEvaluacionNivel.setSelected(false);
        }
        chkSalaPrivada.setDisable(!esClaseIndividual);
        if (!esClaseIndividual) {
            chkSalaPrivada.setSelected(false);
        }
        txtIntegrantes.setDisable(!esClaseGrupal);
        if (!esClaseGrupal) {
            txtIntegrantes.clear();
```

```
        }
    }

    private void limpiarFormulario() {
        txtCodigoClase.clear();
        txtDuracionMeses.clear();
        txtCupoMaximo.clear();
        cbTipo.setValue(null);
        actualizarCamposPorTipo();
    }

    private void refrescarTabla() {
        datosTabla.setAll(repositorio.listar());
    }

    private void mostrarExito(String mensaje) {
        lblMensaje.setStyle(COLOR_OK);
        lblMensaje.setText(mensaje);
    }

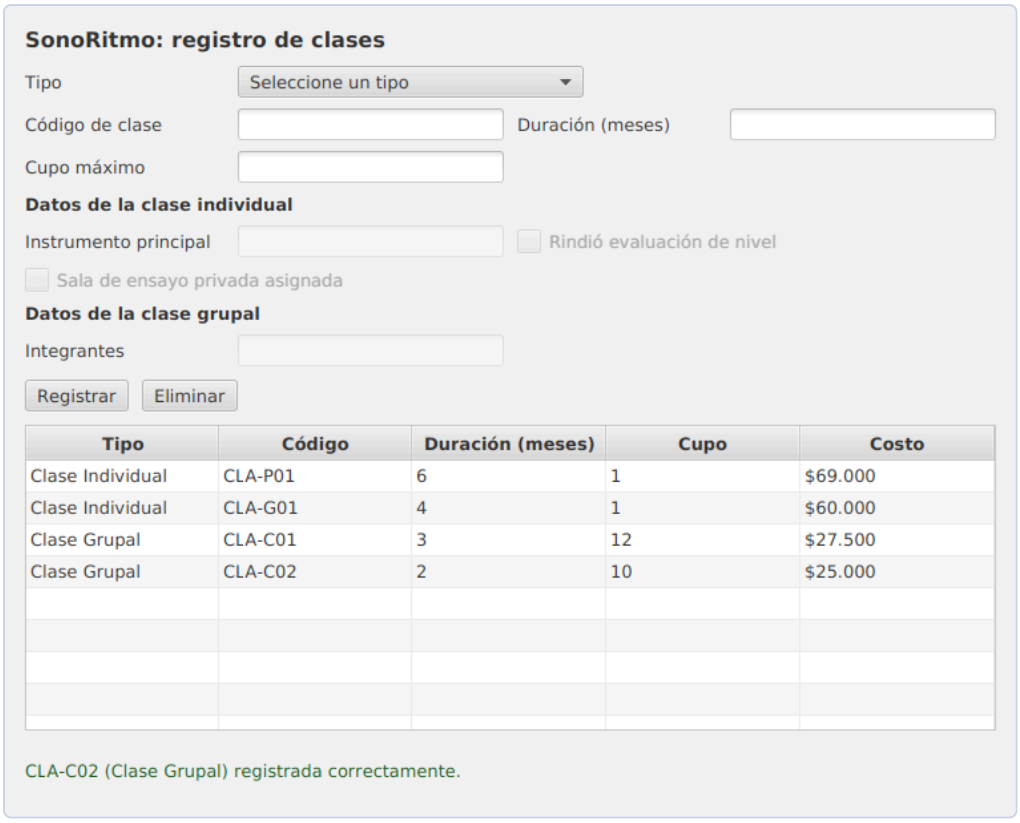
    private void mostrarError(String mensaje) {
        lblMensaje.setStyle(COLOR_ERROR);
        lblMensaje.setText(mensaje);
    }
}
```

## Comprobación realizada

La solución se compiló y ejecutó con los datos de la guía; el resultado de cada paso coincide con lo esperado:

| Tipo | Código de clase | Duración (meses) | Cupo máximo | Costo calculado |
|---|---|---|---|---|
| Clase Individual | CLA-P01 | 6 | 1 | $69.000 |
| Clase Individual | CLA-G01 | 4 | 1 | $60.000 |
| Clase Grupal | CLA-C01 | 3 | 12 | $27.500 |
| Clase Grupal | CLA-C02 | 2 | 10 | $25.000 |

- Eliminar sin selección → aviso en la ventana y sin cambios.
- Duración (meses) = 15 → el mensaje nombra el campo y se conservan los valores.
- Eliminar CLA-P01, cerrar y reabrir → la tabla muestra solo los tres restantes; los JSON coinciden con la tabla.
- Sin archivos JSON → la aplicación inicia con la tabla vacía y sin mensajes.




*Ventana de la solución con los cuatro registros.*

Solución propuesta como material de estudio. Existen otras formas válidas de resolver el caso.